Comparative Analysis of Machine Learning Models for Credit Card Fraud Detection on Imbalanced Datasets

A Report submitted to
Tetso College
Affiliated to
Nagaland University
for award of the degree

of

Bachelor of Computer Applications (BCA)

By

Hamish Choudhury

Registration No: 23150401 of 2023

University Roll No: P23150008

Department of Computer Science
Tetso College, Sovima
May 2025

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "Comparative Analysis of Machine Learning Models for Credit Card Fraud Detection on Imbalanced Datasets" was submitted by Hamish Choudhury, Roll No P23150008 to Tetso College affiliated with Nagaland University, in partial fulfillment of the requirement for Major Project of the 6th Semester of the Bachelor of Computer Applications. It is a bonafide record of the project work carried out by him under my supervision during the semester from January 2026 to June 2026.

(Signature of the Supervisor)

Sir Nangshiba

Assistant Professor

School of Computer Science and Skill Development

Tetso College

Date: .....

CERTIFICATE

This is to certify that the project report entitled "Comparative Analysis of Machine Learning Models for Credit Card Fraud Detection on Imbalanced Datasets" was submitted by Hamish Choudhury, Roll No P23150008, to Tetso College, Dimapur, Nagaland, in partial fulfillment of the requirement for the Major Project of the 6th Semester of the Bachelor of Computer Applications. It is a bonafide record of the project work carried out by him during the semester January 2026 to April 2026.

Miss Rani Majaw

Department Coordinator
Department of Computer Science
and Skill Development

Date: .....

Mr. Talinungsang Lemtur

Dean, School of Computer Science
and Skill Development

Date: .....

DECLARATION

I hereby declare that the project work entitled "Comparative Analysis of Machine Learning Models for Credit Card Fraud Detection on Imbalanced Datasets" submitted to Tetso College, Dimapur, Nagaland, in partial fulfillment of the requirement for Major Project of the 6th Semester of Bachelor of Computer Applications is an original work done by me under the guidance of Sir Nangshiba (Assistant Professor, Department of Computer Science, School of Computer Science and Skill Development, Tetso College) and has not been submitted for the award of any degree.

(Signature of the student)

Hamish Choudhury

Roll No: P23150008

Department of Computer Science

School of Computer Science and Skill Development

Tetso College

ACKNOWLEDGEMENT

I would like to take this opportunity to thank everyone who helped and supported me while working on this project.

First and foremost, I am truly grateful to my supervisor, Sir Nangshiba, for guiding me every step of the way. His advice, patience, and encouragement made a big difference in how this project turned out.

I would also like to thank Miss Rani Majaw, our Department Coordinator, and Mr. Talinungsang Lemtur, Dean of the School of Computer Science and Skill Development, for their support and encouragement throughout. I am also thankful to all the teachers and staff in the Department of Computer Science at Tetso College for creating such a helpful learning environment. I am equally grateful to Kaggle and the IEEE Computational Intelligence Society for making the fraud detection dataset freely available, as it was the foundation of this entire project.

Last but not least, I want to thank my family and friends for always being there for me and keeping me motivated through the tough times.

Hamish Choudhury

ABSTRACT

Credit card fraud is becoming a bigger problem every year. As more people shop online and use digital payments, criminals are finding new ways to steal money. Banks and payment companies lose billions of dollars to fraud each year, and ordinary people also suffer when their cards are compromised. This makes building a reliable system to catch fraudulent transactions as early as possible very important.

One of the biggest challenges in detecting fraud is that real transaction data is very lopsided — for every one fraudulent transaction, there are roughly 28 legitimate ones. This means that a simple computer program that labels every transaction as "not fraud" would still be right 96.5% of the time, even though it catches zero fraud. This project tackles that problem head-on.

In this project, I tested and compared eight different machine learning models to see which one is best at spotting fraud. I used the IEEE-CIS Fraud Detection dataset, which contains 590,540 real online transactions collected over six months, with about 3.5% of them being fraudulent. The eight models tested were: Logistic Regression, SGD Logistic Regression, Random Forest, XGBoost, LightGBM, a neural network (MLP) in static mode, the same neural network in adaptive mode, and a combined Stacked Hybrid model.

Before feeding the data into the models, I cleaned and prepared it carefully. I filled in missing values, converted text fields into numbers the models can understand, created useful new features like the time of day a transaction happened and whether the amount was a round number, and compressed a large group of 339 features down to just 50 using a technique called PCA. I also used special methods to make sure the models did not simply ignore fraud because there is so little of it in the data.

To measure how well each model performed, I used three scores: ROC-AUC (how good the model is at separating fraud from non-fraud overall), PR-AUC (how well it finds fraud without raising too many false alarms), and Recall@5%FPR (how many actual fraud cases it catches before generating too many false alerts). The results showed that the Stacked Hybrid and Random Forest models performed best overall. The adaptive neural network was the most interesting result — by continuously learning from new transactions and detecting

when fraud patterns changed, it was better at catching fraud under realistic conditions than the static version.

The main takeaway is that combining multiple models together gives the best results, and that having a model which can adapt over time is very valuable in the real world where fraud patterns keep changing.

Keywords: Credit Card Fraud Detection, Machine Learning, Deep Learning, Class Imbalance, IEEE-CIS Dataset, XGBoost, Random Forest, MLP, Concept Drift, ADWIN, Stacking Ensemble

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Background

Every time someone makes an online purchase or taps their card at a store, there is a small chance that transaction could be fraudulent. As digital payments have grown explosively over the past decade, so has the opportunity for criminals to exploit them. Credit card fraud now costs the global economy billions of dollars every year, hurting banks, businesses, and everyday customers.

For a long time, banks tried to catch fraud by setting up rigid rules — for example, flagging any transaction over a certain amount or from an unusual location. But fraudsters are smart and quick to adapt. As soon as they figure out the rules, they find ways around them. This means rule-based systems need constant manual updates, which is expensive and slow.

Machine learning offers a smarter approach. Instead of hard-coded rules, machine learning models learn from thousands of past transactions to recognize patterns that are typical of fraud. However, there is a catch: in any real dataset, fraudulent transactions are a tiny minority — often less than 5% of all records. This creates a "class imbalance" problem, where a lazy model can score very high accuracy just by calling everything legitimate. Building a model that actually catches fraud — without overwhelming investigators with false alarms — requires careful thinking about how to train and evaluate it.

This project takes on that challenge. I compared eight different machine learning models on a large, real-world fraud dataset, using evaluation methods designed specifically for imbalanced data. I also built an adaptive model that can adjust itself over time as fraud patterns shift — something very important for real-world deployment.

1.2 Aim of the Project

The aim of this project is to find out which machine learning approach works best for detecting credit card fraud when the data is highly imbalanced. I wanted to not just pick a winner, but understand why different models perform differently, and what techniques help them handle the imbalance problem better.

1.3 Objectives of the Project

The specific things I set out to do in this project are:

- To clean and prepare the IEEE-CIS Fraud Detection dataset, filling in missing data, converting text into numbers, and reducing the number of features to make training faster.
- To train six baseline models: Logistic Regression, SGD Logistic Regression, Random Forest, XGBoost, LightGBM, and a static neural network (MLP).
- To build an Adaptive MLP that can detect when fraud patterns are changing and update itself accordingly, using a technique called ADWIN drift detection.
- To create a Stacked Hybrid model that combines Random Forest, XGBoost, and MLP into one stronger model.
- To evaluate all models using metrics that are fair for imbalanced data: ROC-AUC, PR-AUC, and Recall@5%FPR.
- To discuss the trade-offs between each model — such as how accurate they are, how fast they train, and how easy they are to understand.

1.4 Scope of the Study

This project focuses specifically on detecting credit card fraud from transaction data, using supervised machine learning. I only used the IEEE-CIS dataset and did not look at other types of fraud like insurance fraud. I did not spend time fine-tuning every model to its absolute best settings — the goal was a fair comparison across different approaches. The adaptive streaming part of the project simulates a real situation where fraud labels only become available after a delay, just like in real banking systems.

1.5 Organization of the Report

The rest of this report is laid out as follows. Chapter 2 reviews what other researchers have done in this area. Chapter 3 describes the dataset and how I prepared it. Chapter 4 explains how each model was built. Chapter 5 presents the results and what they mean. Chapter 6 covers how the models were tested and the limitations of the testing process. Chapter 7 wraps up with conclusions and ideas for future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Credit Card Fraud Detection

Researchers have been working on fraud detection for more than two decades. In the early days, the main approach was to write rules by hand — things like "flag any transaction over $5,000" or "block purchases from certain countries." These worked reasonably well for a while, but criminals adapted quickly, and keeping the rules up to date became a constant battle.

As datasets grew larger and computers got faster, machine learning became the preferred approach. Instead of humans writing rules, algorithms learn from examples of past fraud and apply that knowledge to new transactions. A landmark survey by Abdallah et al. (2016) showed that supervised learning — where the model learns from labeled examples of both fraud and legitimate transactions — consistently outperforms other approaches when enough labeled data is available.

2.2 Machine Learning Approaches

2.2.1 Tree-Based Ensemble Methods

Among all the machine learning methods tried for fraud detection, tree-based models have consistently come out on top. Two models in particular — XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017) — have dominated Kaggle competitions and industry benchmarks. These models work by building many small decision trees and combining their answers, which makes them both accurate and resilient to noisy data. They also handle missing values naturally, which is a big advantage when dealing with messy real-world data like financial transactions.

Random Forest (Breiman, 2001) is another popular tree-based method that works by building many independent trees and voting on the final answer. It is slightly less accurate than boosting methods in many comparisons but is more stable and less likely to overfit.

2.2.2 Logistic Regression

Logistic regression is one of the oldest and simplest classification methods. It is fast to train and easy to interpret, which is why it is often used as a starting point for comparison. However, it works by drawing a straight line to separate fraud from non-fraud, which is too simplistic for the complex patterns found in real transaction data. Studies like Dal Pozzolo et al. (2015) consistently show that logistic regression falls well short of tree-based methods on fraud detection tasks.

2.3 Deep Learning for Fraud Detection

Deep learning — which involves training large neural networks with many layers — has become increasingly popular for fraud detection in recent years. Neural networks can learn very complex patterns that simpler models miss, but they also require more data and computing power to train well.

Fiore et al. (2019) showed that neural networks called autoencoders can learn what "normal" transactions look like, and then flag anything that deviates significantly as potential fraud. Wang et al. (2018) used recurrent neural networks, which are designed to work with sequences of events, to capture patterns in how a cardholder's spending changes over time.

One particularly useful technique is Monte Carlo Dropout (Gal and Ghahramani, 2016). Normally, neural networks give a single prediction for each transaction. With Monte Carlo Dropout, the model makes many slightly different predictions for the same transaction by randomly switching off parts of the network each time. The spread of these predictions tells us how confident the model is — a transaction with very inconsistent predictions across runs is one the model is unsure about, which could be a signal worth investigating.

2.4 Handling Class Imbalance

The fact that fraud is so rare in the data — often less than 4% of all transactions — is one of the core difficulties in building a fraud detection model. This is called the class imbalance problem, and it causes many standard machine learning techniques to effectively ignore the minority class (fraud).

Several strategies have been developed to address this. SMOTE (Chawla et al., 2002) works by creating artificial fraud examples that are similar to real ones, effectively balancing

the dataset. Cost-sensitive learning penalizes the model more heavily for missing a fraudulent transaction than for flagging a legitimate one as suspicious.

Focal Loss (Lin et al., 2017), originally invented for detecting small objects in images, works by making the model focus more attention on the transactions it finds hardest to classify correctly. This is effective in fraud detection because the vast majority of transactions are straightforward legitimate ones, and the model needs to be pushed to pay more attention to the rare fraud cases.

Another practical technique is WeightedRandomSampler, which is available in the PyTorch library. Instead of duplicating fraud examples in the dataset, it simply picks more fraud examples per batch during training, helping the model see proportionally more fraud without artificially inflating the dataset size.

2.5 Concept Drift and Adaptive Learning

One challenge that is often overlooked in research but is very important in the real world is concept drift. This simply means that fraud patterns change over time. Criminals discover new tricks, exploit new vulnerabilities, and adapt their behaviour to avoid detection. A model trained on data from six months ago may perform poorly on today's transactions because the fraud it was trained to detect no longer looks the same.

ADWIN (Bifet and Gavalda, 2007) is a well-known method for detecting when this kind of drift is happening. It monitors the model's error rate over a sliding window of recent transactions, and raises an alarm when the error rate changes significantly — which is a sign that the fraud patterns have shifted and the model needs to be updated.

Online learning frameworks like the River library allow models to update themselves continuously as new data arrives, rather than waiting for a scheduled retraining cycle. This project combines deep learning with ADWIN drift detection to build an adaptive model that can keep up with changing fraud patterns in real time.

2.6 Stacking Ensembles

Stacking (Wolpert, 1992) is a technique for combining multiple models into one. The idea is simple: instead of picking one best model, you let several different models make their predictions, and then train a second-level model to combine those predictions into a final

answer. The second-level model learns which base models are most reliable in which situations, potentially getting better results than any single model alone. Stacking has won many machine learning competitions and is widely used in industry applications.

2.7 Existing Systems and Their Drawbacks

2.7.1 Existing Systems

Most fraud detection systems used by banks and payment companies today combine two things: a set of manually written rules (like blocking transactions from known fraudulent merchants) and a machine learning model that scores each transaction for risk. These systems work reasonably well for catching known types of fraud.

2.7.2 Drawbacks

- The manual rules need to be updated constantly, which takes time and human expertise.
- The machine learning models are typically static — once trained, they do not update themselves as fraud patterns change, so their performance gradually degrades.
- Standard accuracy scores are misleading in this setting because a model that calls every transaction legitimate would still be 96.5% accurate on this dataset.
- Most deployed models cannot tell you how confident they are in a given prediction, making it hard for analysts to prioritize which flagged transactions to review first.
- Neural network models are often seen as "black boxes" — they produce a score but do not explain why, which makes regulators and compliance teams uncomfortable.

2.8 Proposed System and Advantages

This project proposes a more comprehensive approach that includes multiple models from different families, a neural network that estimates its own uncertainty, and an adaptive component that updates itself when fraud patterns change. The key advantages over existing systems are:

- A fair comparison across eight different model types, giving a clear picture of which approaches work best for this kind of problem.

- An adaptive model that can detect when fraud patterns are shifting and update itself without needing a full retrain.
- Uncertainty-aware predictions that help analysts decide which flagged transactions need the most urgent attention.
- Evaluation using metrics that are actually meaningful for fraud detection, not just overall accuracy.
- A fully reproducible implementation that could be adapted and deployed as a real fraud detection system.

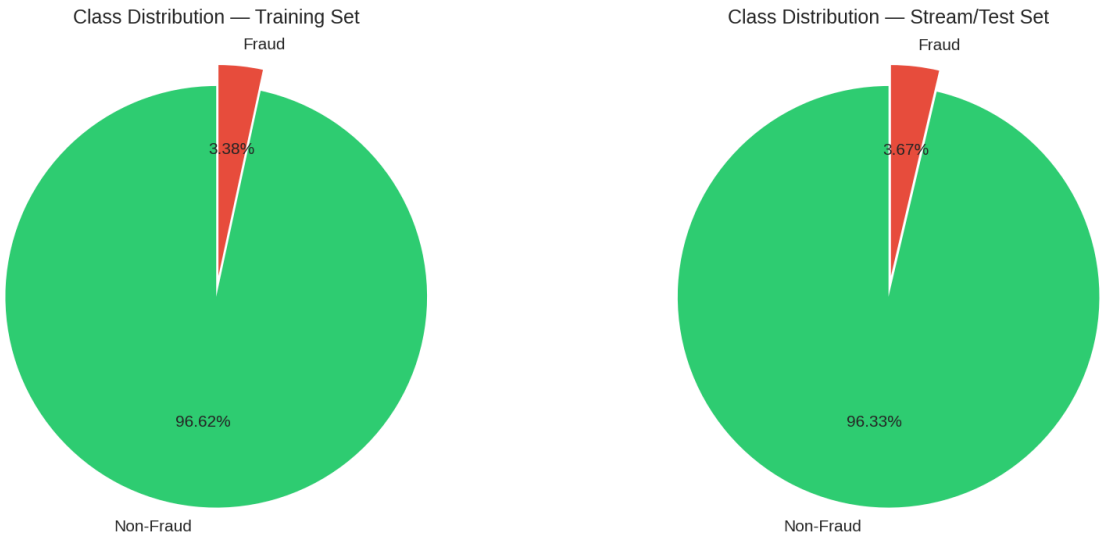

Figure 2.1: Class Distribution in Training and Stream/Test Sets

Figure 2.1 shows just how imbalanced the dataset is. In the training set, only 3.38% of transactions are fraud, and in the test set, the figure is 3.67%. The vast green area in each pie chart represents legitimate transactions. This imbalance is why standard accuracy is not a useful metric for this project, and why special techniques are needed to handle the minority class.

CHAPTER 3: DATASET AND PREPROCESSING

3.1 Dataset Description

For this project, I used the IEEE-CIS Fraud Detection dataset, which was released through a Kaggle competition run by the IEEE Computational Intelligence Society together with Vesta Corporation, a company that processes online payments. This dataset is one of the most realistic and widely studied benchmarks for fraud detection research because it comes from real transactions rather than being artificially generated.

As shown in table 3.1, the dataset contains 590,540 card-not-present transactions (meaning transactions where the physical card was not used, such as online purchases) collected over six months. Each transaction is tagged as either fraudulent or legitimate. The data comes in two parts: a transaction file with 394 pieces of information per transaction, and an identity file with 37 additional fields. These two files are linked by a transaction ID. Not every transaction has identity information — only about 24% of transactions have a matching identity record.

Table 3.1: Dataset Statistics

Attribute	Value
Total Transactions	590,540
Fraudulent Transactions	20,663 (3.5%)
Legitimate Transactions	569,877 (96.5%)
Total Features (raw)	431
Transaction File Features	394
Identity File Features	37
Collection Period	6 months
Training Set Size (60%)	354,324
Streaming/Test Set Size (40%)	236,216

3.2 Feature Groups

The 431 features in the dataset can be grouped into several categories as shown in Table 3.2.

Table 3.2: Feature Groups in the IEEE-CIS Dataset

Feature Group	Description	Count
TransactionDT	Time since a fixed reference point	1
TransactionAmt	Payment amount in USD	1
ProductCD	Type of product purchased	1
card1-card6	Card details (type, bank, country, etc.)	6
addr1, addr2	Billing address information	2
dist1, dist2	Distance-related features	2
P_emaildomain, R_emaildomain	Email domains of buyer and recipient	2
C1-C14	Anonymized counting features	14
V1-V339	Anonymized Vesta behavioral features	339
DeviceType, DeviceInfo	Device used for the transaction	2
id_01-id_38	Identity-related information	38

3.3 Data Splitting Strategy

One of the most important decisions in this project was how to split the data into training and test sets. I used a chronological split, meaning I sorted all transactions by time and used the first 60% (354,324 transactions) for training and the remaining 40% (236,216 transactions) for testing. This is the right approach for fraud detection because it mimics how a real system would work — you train on past data and test on future data.

If I had randomly shuffled and split the data instead, information from future transactions could accidentally leak into the training process, making results look better than they really are. By keeping the split in time order, I make sure the results reflect how the models would actually perform in a real deployment.

For the neural network model, I further divided the training set 80/20 to create a small validation portion for monitoring performance during training. All data preparation steps (like scaling numbers and converting categories to numbers) were done using only the training data, and then applied to the test data. This ensures the test data truly represents unseen future information.

3.4 Preprocessing Pipeline

Raw transaction data cannot be fed directly into a machine learning model. It needs to be cleaned, transformed, and organized. Table 3.3 summarizes the steps I took.

Table 3.3: Preprocessing Steps Summary

Step	Method	Details
Fill Missing Values	Median / "missing" label	Numbers: replaced with column median; Text: replaced with the word "missing"
Encode Categories	Target Encoding	Each category replaced by the fraud rate for that category, calculated from training data only
Extract Time Features	TransactionDT breakdown	Hour of day, day of week, and day of month extracted
Extract Amount Features	Log and flags	Log of amount, decimal portion, and flag for round-number amounts
Card-level Aggregation	Group-by statistics	Per-card fraud rate and transaction count added as new features

Step	Method	Details
Reduce V-columns	PCA (339 down to 50)	Keeps about 95% of the information in far fewer features
Scale Numbers	StandardScaler	Fitted on training data only, then applied to test data

3.4.1 Filling in Missing Values

Out of the 431 raw features, 414 have at least some missing values. The V-columns and identity features are the worst affected. For numeric columns, I replaced missing values with the median of that column in the training data. For text columns, I simply used the word "missing" as a category, which the encoding step then handled like any other category.

3.4.2 Converting Text to Numbers (Target Encoding)

Machine learning models work with numbers, not text. Features like the type of product purchased, the email domain, or the type of device used are text values with many possible options. I used a technique called target encoding, which replaces each text option with the average fraud rate for transactions that had that same value. For example, if 8% of all transactions from a particular email domain turned out to be fraud in the training data, every transaction with that email domain gets the value 0.08. This approach preserves useful fraud-related information while turning text into numbers.

3.4.3 Creating New Features

I also created several new features that were not in the original dataset but could help models spot fraud. From the timestamp, I extracted the hour of the day, the day of the week, and the day of the month — because fraud tends to happen more at certain times. From the transaction amount, I calculated its logarithm (which spreads out large values more evenly), the decimal portion of the amount, and a flag for whether the amount was a round number like $100 or $500. I also computed each card's historical fraud rate and transaction count from the training data and added these as new features, since a card with a high past fraud rate is a strong warning sign.

3.4.4 Shrinking the V-Columns with PCA

The 339 V-columns are anonymized behavioral features created by Vesta. Many of them contain very similar information. To avoid redundancy and speed up training, I used Principal Component Analysis (PCA) to compress these 339 features down to just 50. These 50 new features, called principal components, capture about 95% of the meaningful variation in the original 339, while being much more compact. This reduced the overall number of features from around 381 down to about 92.

3.5 Handling the Imbalance Problem

Because fraud accounts for only 3.5% of transactions, there is a risk that models will simply learn to ignore it. I used different strategies for different types of models. For tree-based models like Random Forest and LightGBM, I set a "class weight" parameter that tells the model to treat each fraud case as if it were 28 times more important than a legitimate transaction. For XGBoost, a similar parameter called scale_pos_weight was used. For the neural network, I used a technique called WeightedRandomSampler to show the model more fraud examples per training batch, and also applied a modified loss function that punishes the model more heavily for missing fraud than for raising a false alarm.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture Overview

The fraud detection system I built has three main parts. First, a data preparation pipeline that takes raw transaction records and transforms them into clean, model-ready data. Second, a training and evaluation module where all eight models are trained and their performance is measured. Third, an adaptive streaming module that simulates a live transaction stream, where the neural network updates itself as new data comes in and fraud patterns shift.

The entire system was coded in Python, using well-known libraries including scikit-learn, XGBoost, LightGBM, and PyTorch. The adaptive streaming part relies on the River library for drift detection. The project was run on Google Colab using a free GPU, and all the trained models were saved so they could be used in a Streamlit web application for demonstration.

4.2 Baseline Models

4.2.1 Logistic Regression

Logistic Regression is the simplest model in this comparison. It tries to find a straight boundary that separates fraudulent from legitimate transactions. I used L1 regularization (which helps the model focus on the most important features) and set the class weights to balanced so it pays more attention to fraud. A second version called SGD Logistic Regression was also trained — this version processes the data in small batches, which makes it possible to update it incrementally with new transactions.

4.2.2 Random Forest

Random Forest builds 100 separate decision trees, each trained on a random subset of the data and features. The final prediction is made by taking a majority vote across all 100 trees. This approach is robust to noise and tends to generalize well. I set the maximum tree depth to 10 to prevent overfitting, and used balanced class weights.

4.2.3 XGBoost

XGBoost is one of the most powerful models for structured data like financial records. It builds trees one at a time, with each new tree focusing on correcting the mistakes of the previous ones. I configured it with 100 trees and set the class imbalance weight to approximately 28.6 (the ratio of legitimate to fraudulent transactions in the training data). The trained model was saved in a standard format for later use.

4.2.4 LightGBM

LightGBM is similar to XGBoost but uses a faster training algorithm that works particularly well on large datasets. It achieved very similar results to XGBoost in this project but trained faster. I used the same basic settings: 100 trees, a depth limit of 6, and balanced class weights.

4.3 Deep Learning Model: Improved MLP

The neural network used in this project is a Multi-Layer Perceptron (MLP) with five layers of sizes 256, 128, 64, 32, and 1. Think of it as a series of filters: the first layer processes the raw features, each subsequent layer refines and compresses the information, and the final layer outputs a single fraud probability score. Between each layer, I applied dropout — a technique that randomly switches off 30% of the connections during each training step. This forces the network to learn more robust patterns and also enables Monte Carlo Dropout at test time.

Monte Carlo Dropout works by running the network many times (20 times in this project) with dropout still switched on, producing slightly different predictions each time. The average of these predictions is the final fraud score, and the spread of predictions is the uncertainty estimate. A transaction where the model gives very different scores across the 20 runs is one the model is unsure about.

The network was trained using the AdamW optimizer, which includes a weight decay term to prevent overfitting. I used a specialized loss function that penalizes the model more for missing fraud than for false alarms. Training automatically slowed down the learning rate when progress stalled, and stopped early if performance on the validation set did not improve for 5 consecutive rounds.

Table 4.1: Model Hyperparameter Settings

Model	Key Settings
Logistic Regression	C=0.1, L1 penalty, SAGA solver, balanced class weights
SGD Logistic Regression	Log loss, L1 penalty, balanced weights, 10 training passes
Random Forest	100 trees, max depth 10, balanced class weights
XGBoost	100 trees, max depth 6, fraud weight factor ~28.6
LightGBM	100 trees, max depth 6, learning rate 0.1, balanced weights
MLP Static	5 layers (256-128-64-32-1), 30% dropout, AdamW optimizer
MLP Adaptive	Same as MLP Static, plus ADWIN drift detection and replay buffer
Stacked Hybrid	Random Forest + XGBoost + MLP combined through Logistic Regression

4.4 MLP Training Progress

Figure 4.1 shows how the neural network's training error changed over 20 training rounds (called epochs). The error dropped sharply in the first few rounds, then continued to fall more gradually. The smooth decline without any spikes indicates that training was stable and the model was learning properly without becoming overconfident on the training data.

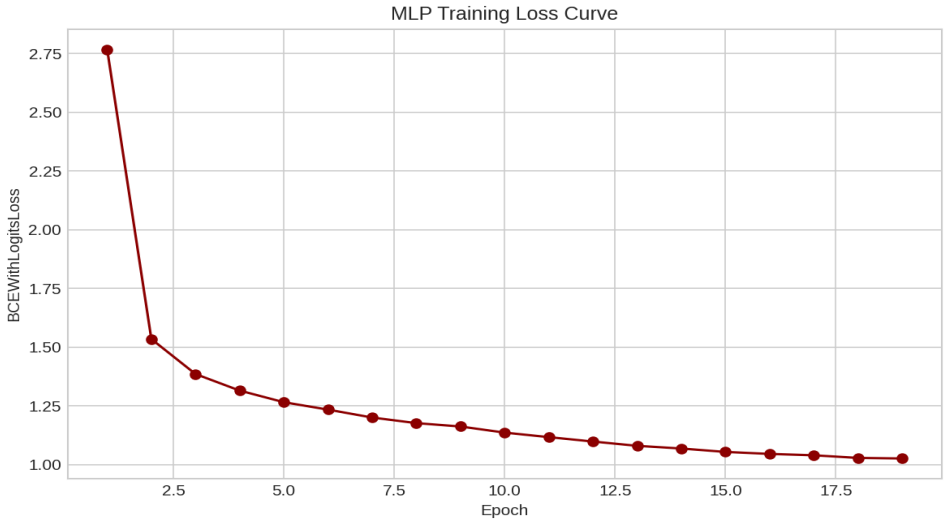

Figure 4.1: MLP Training Loss Curve

4.5 Adaptive Streaming Module

The adaptive streaming module is the most novel part of this project. It simulates what would happen if the trained neural network were deployed in a real bank that processes transactions one by one.

Each transaction is fed into the model, which produces both a fraud score and an uncertainty estimate. The model waits 1,000 transactions before learning from the labels (because in real life, confirming whether a transaction was actually fraud takes time). When a label becomes available, it is fed into the ADWIN drift detector. If ADWIN detects that fraud patterns have changed, or if the model was particularly uncertain about a transaction, that transaction gets added to a replay buffer. When enough uncertain or drift-affected samples have accumulated (64 in this project), the model does a mini update on those samples to bring itself up to date with the current fraud patterns.

4.6 Stacked Hybrid Model

The Stacked Hybrid model works by using three models — Random Forest, XGBoost, and the neural network — as a team. Each model sees the training data and makes predictions. These predictions are then used as inputs to a second model (a simple Logistic Regression), which learns how to best combine the three individual opinions into one final verdict.

To make sure this combining step is done fairly, the three base models are trained on the first 80% of the training data, and their predictions on the remaining 20% are used to train the Logistic Regression combiner. This way, the combiner never sees the same data the base models were trained on, which prevents it from simply memorizing their outputs.

4.7 Which Features Matter Most?

Figure 4.2 shows the top 20 most important features according to XGBoost. The most important feature by a very wide margin is card1_fraud_rate — the historical fraud rate associated with the payment card used. This makes intuitive sense: if a card has been involved in fraud before, it is much more likely to be used for fraud again. Several of the PCA components of the V-columns also rank highly, along with the counting features C14 and C5, and the flag for whether the transaction amount was a round number.

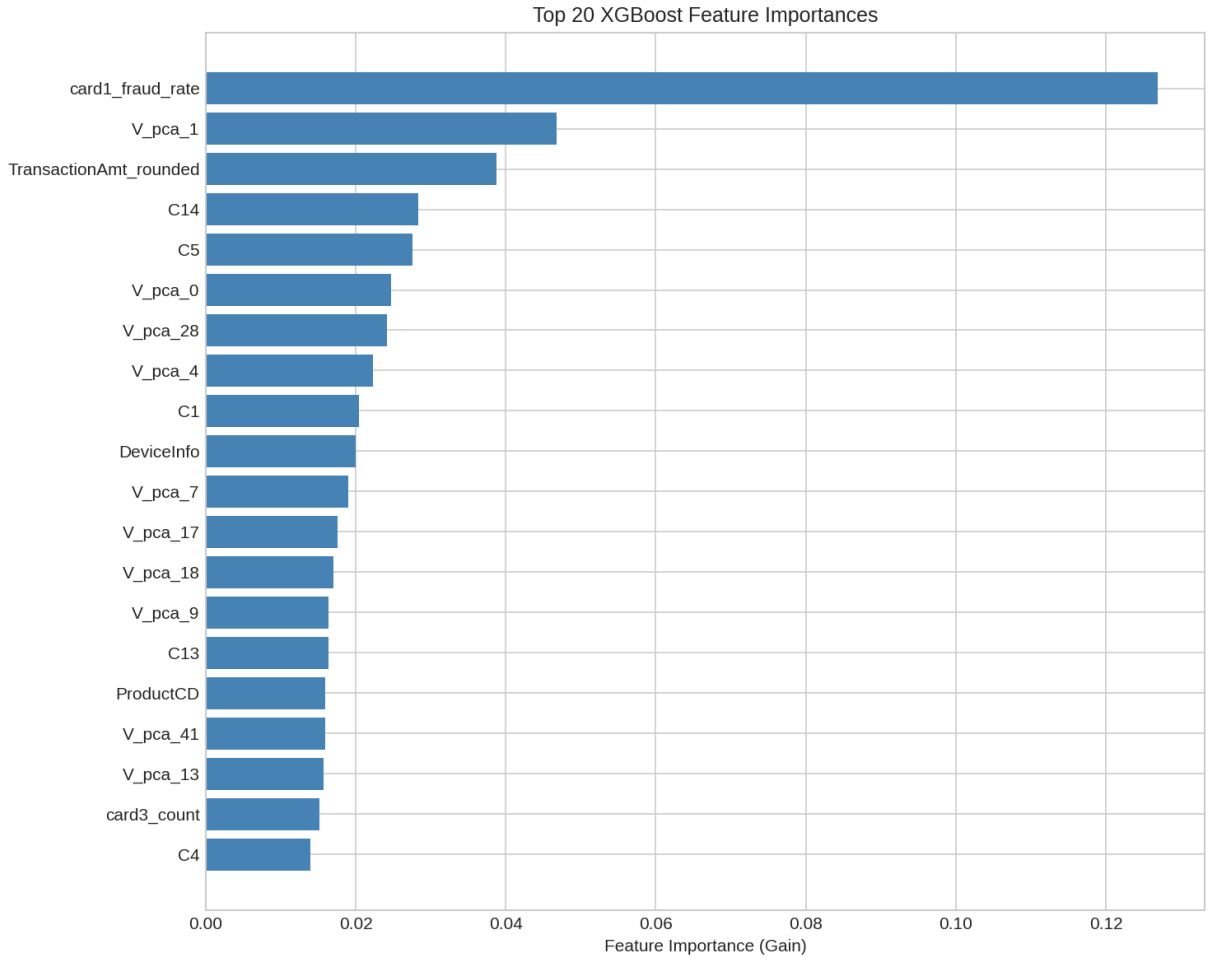

Figure 4.2: Top 20 XGBoost Feature Importances

CHAPTER 5: RESULTS AND DISCUSSION

5.1 How the Models Were Scored

Before looking at results, it is important to understand how I measured performance. I used three different scores, each capturing a different aspect of how good the model is:

- ROC-AUC: This measures how well the model can rank transactions by suspicion level. A score of 1.0 means every fraudulent transaction is ranked above every legitimate one. A score of 0.5 means the model is no better than a random guess. This is a useful overall indicator but can be misleading when fraud is very rare.
- PR-AUC (Precision-Recall AUC): This focuses specifically on the fraud class. It asks: when the model flags a transaction as fraud, how often is it right (precision)? And out of all real fraud cases, how many does it catch (recall)? PR-AUC summarizes this trade-off across all possible thresholds. This was the primary metric in this project because it is most informative when fraud is rare.
- Recall@5%FPR: This answers a very practical question — if we allow the model to raise false alarms on 5% of legitimate transactions (roughly one in twenty), how many actual fraud cases can it catch? This directly reflects how a real fraud team would use the model.

5.2 Overall Results

Table 5.1 and Figure 5.8 show the results for all eight models side by side.

Table 5.1: Model Performance Summary (ROC-AUC, PR-AUC, Recall@5%FPR)

Model	ROC-AUC	PR-AUC	Recall@5%FPR
Stacked Hybrid	86.2%	43.0%	52.2%
Random Forest	86.3%	41.3%	52.6%
XGBoost	80.4%	40.8%	51.4%
LightGBM	81.9%	41.3%	51.6%
MLP Adaptive	82.7%	23.5%	41.3%
MLP Static	79.7%	27.2%	39.2%
Logistic Regression	81.7%	18.6%	37.1%
SGD Logistic Regression	69.7%	11.9%	29.5%

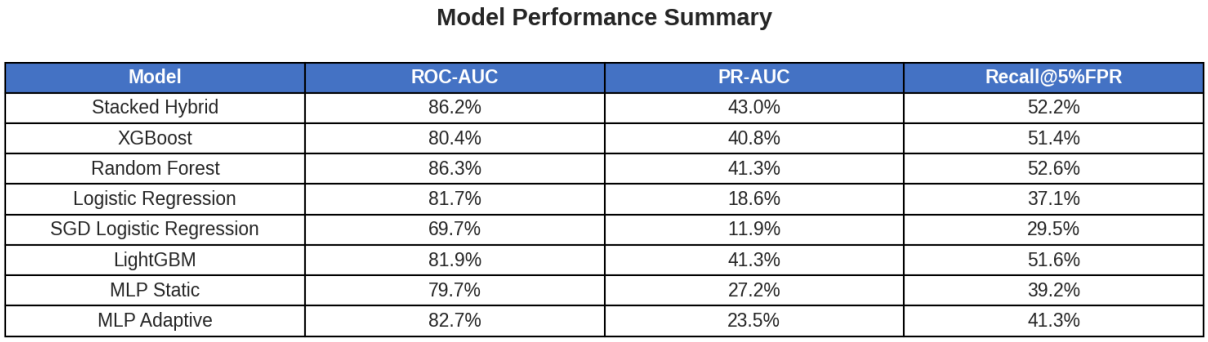

Figure 5.8: Model Performance Summary Table

5.3 ROC-AUC Results

Figure 5.1 shows the ROC-AUC scores for all models. Random Forest and the Stacked Hybrid are essentially tied at the top, with scores of 86.3% and 86.2% respectively. This is a strong result — both models are very good at sorting transactions from most suspicious to least suspicious. LightGBM and Logistic Regression sit in the middle of the pack, and SGD

Logistic Regression brings up the rear at just 69.7%, showing that this simple model struggles significantly with this task.

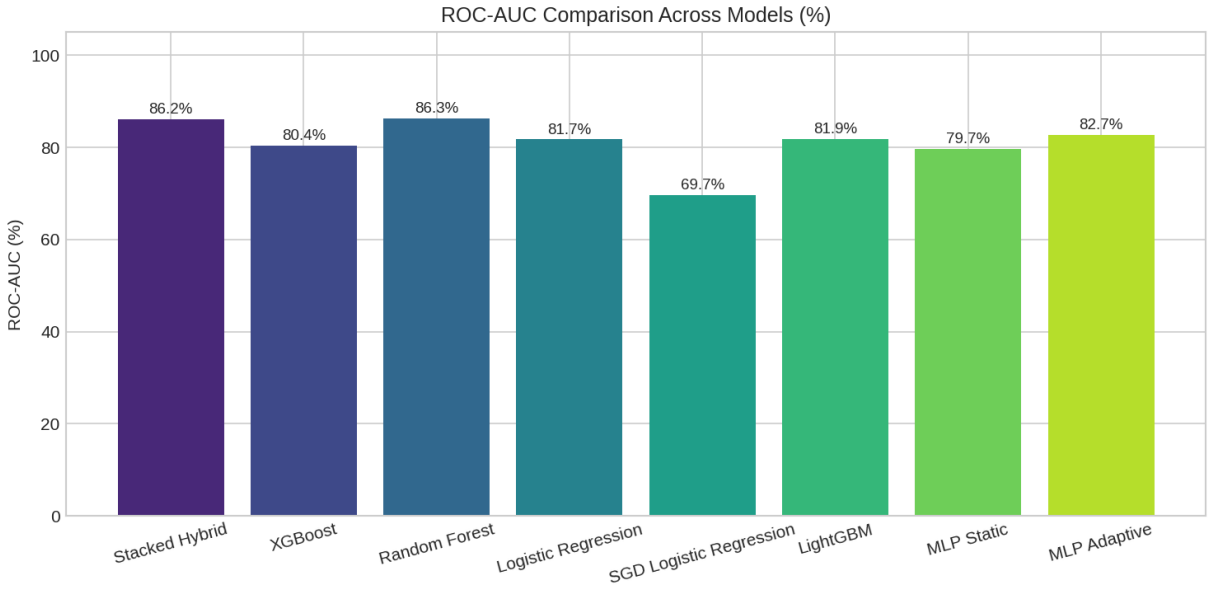

Figure 5.1: ROC-AUC Comparison Across All Models

Figure 5.4 shows the full ROC curves for the static models. The further a curve bows toward the top-left corner, the better. Random Forest's orange curve clearly pulls ahead of the others across most of the chart, confirming its lead on this metric.

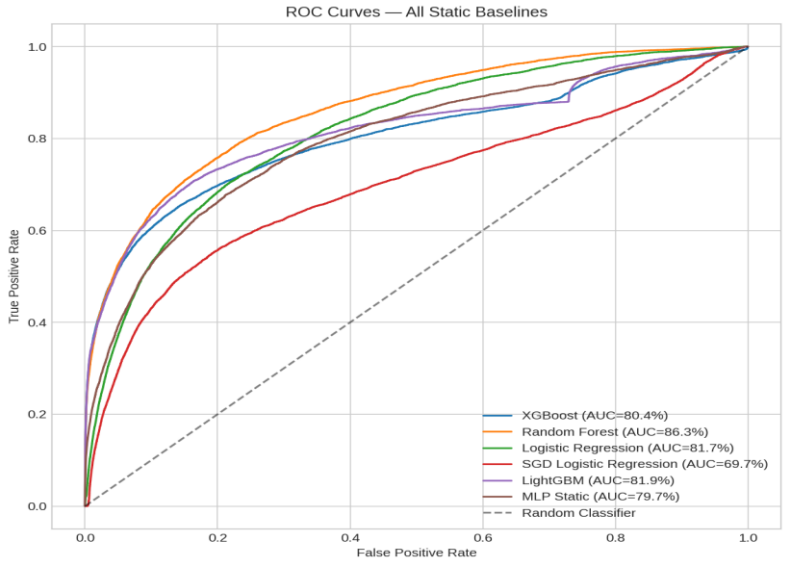

Figure 5.4: Overlaid ROC Curves — All Static Baselines

5.4 PR-AUC Results

PR-AUC tells a more nuanced story. Figure 5.2 shows that the Stacked Hybrid edges ahead of the pack here, scoring 43.0% — slightly higher than Random Forest and LightGBM (both 41.3%) and XGBoost (40.8%). The neural network models score considerably lower: the static MLP at 27.2% and the adaptive MLP at 23.5%. This suggests that while the neural network is competitive at ranking transactions overall, tree-based models are better at maintaining high precision when recall is pushed higher.

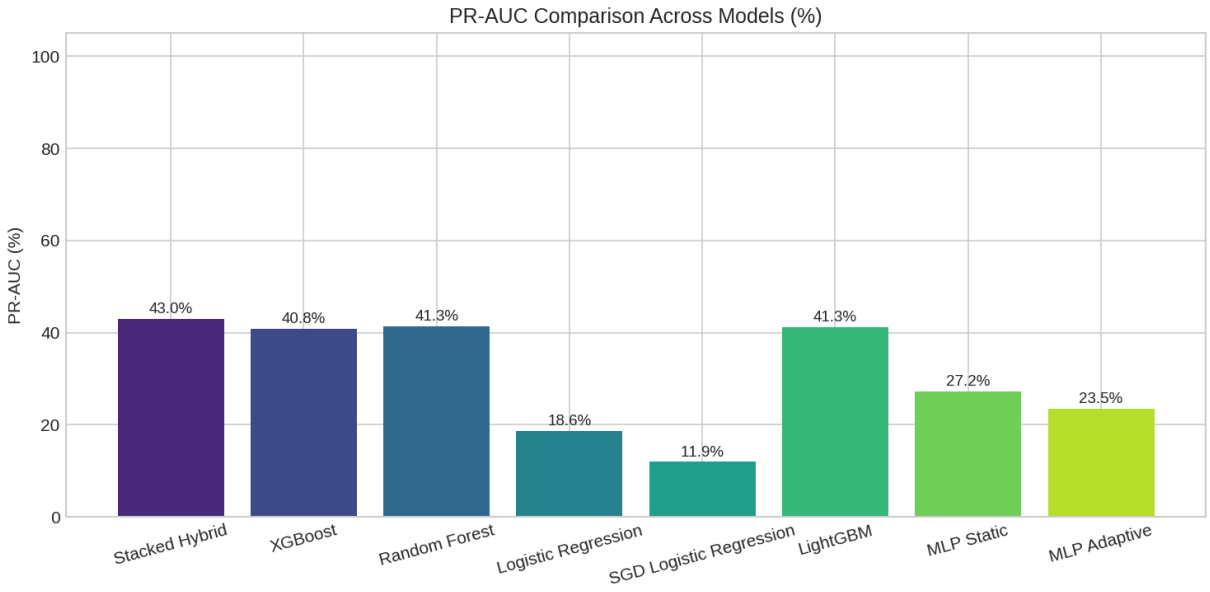

Figure 5.2: PR-AUC Comparison Across All Models

Figure 5.5 shows the precision-recall curves for all models. The curves for XGBoost, Random Forest, and LightGBM all stay high for a long stretch before dropping, showing that these models can catch a lot of fraud before their false alarm rate gets out of hand. The red SGD Logistic Regression curve barely rises above the baseline, highlighting just how poorly this model handles the imbalanced dataset.

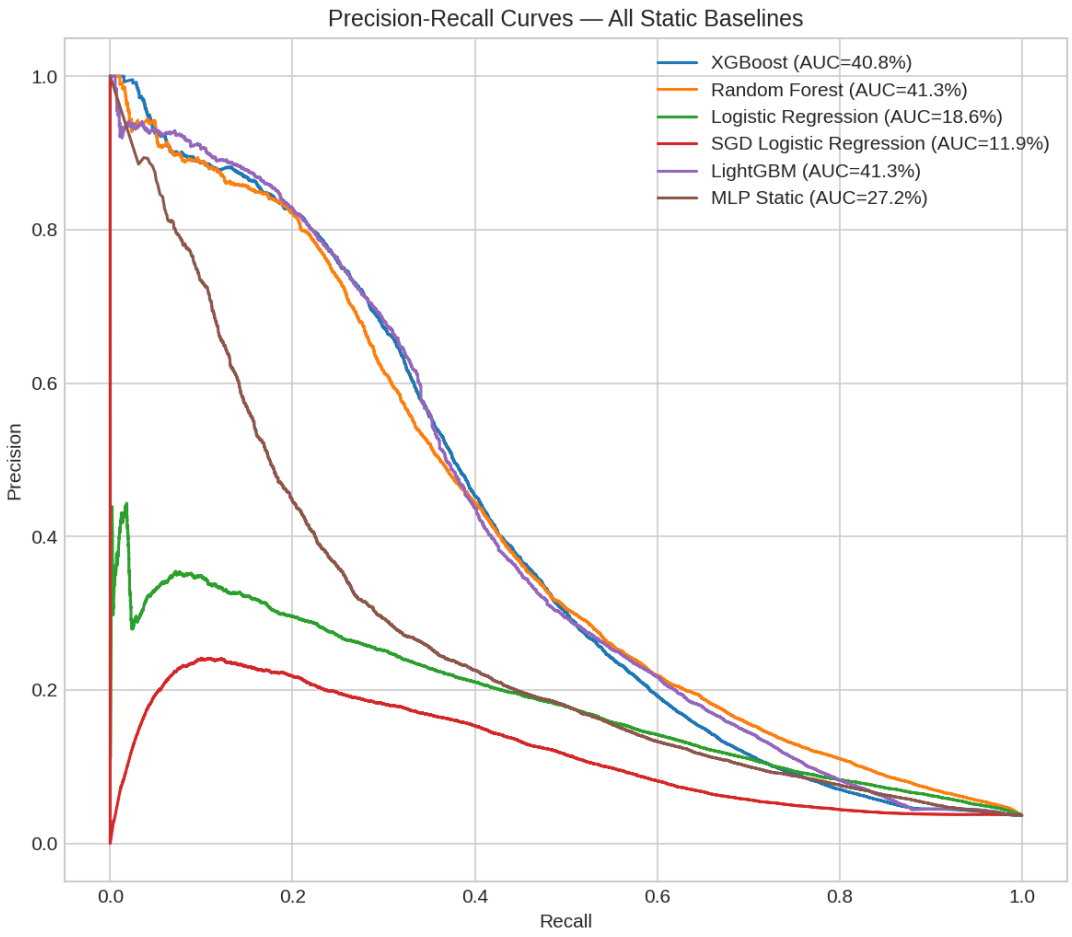

Figure 5.5: Precision-Recall Curves — All Static Baselines

5.5 Recall@5%FPR Results

Figure 5.3 shows how many fraud cases each model catches when allowed to flag up to 5% of legitimate transactions as suspicious. Random Forest catches the most at 52.6%, with the Stacked Hybrid very close behind at 52.2%. In practical terms, this means these models can catch just over half of all fraud cases while keeping false alarms to a manageable level. An interesting finding here is that the Adaptive MLP (41.3%) outperforms the Static MLP (39.2%), suggesting that the model's ability to update itself when fraud patterns change does pay off in this specific metric.

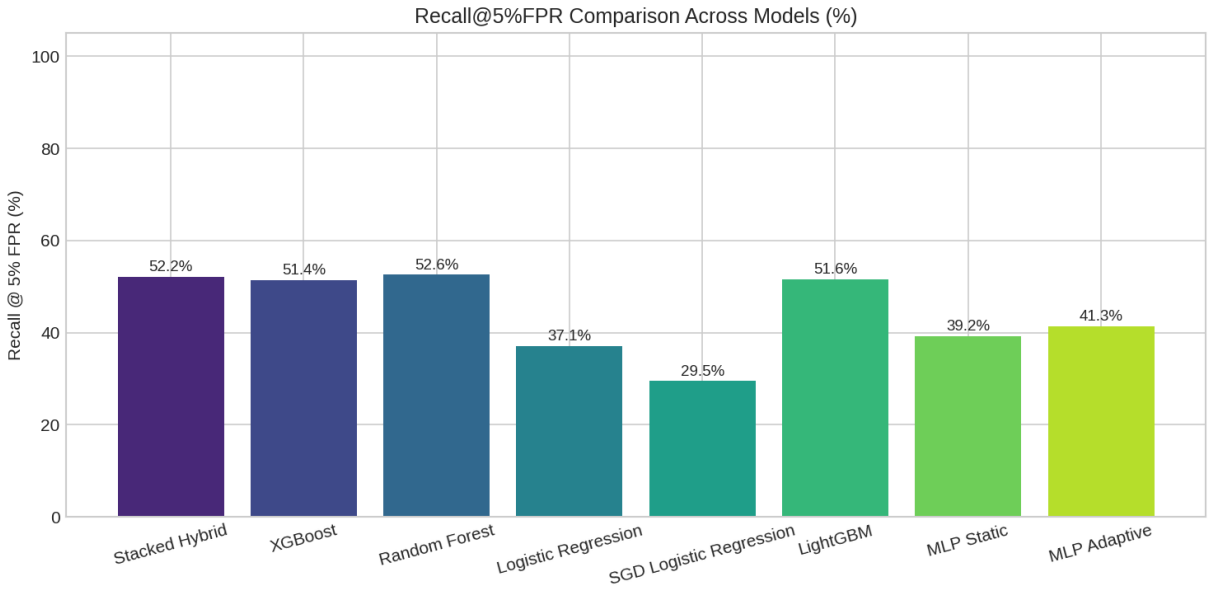

Figure 5.3: Recall@5%FPR Comparison Across All Models

5.6 Looking at the Confusion Matrices

Figure 5.6 shows the confusion matrices for XGBoost, Random Forest, and the static MLP. A confusion matrix breaks down the model's predictions into four categories: correctly identified fraud (top-right), correctly identified legitimate transactions (top-left), missed fraud (bottom-left), and false alarms (bottom-right).

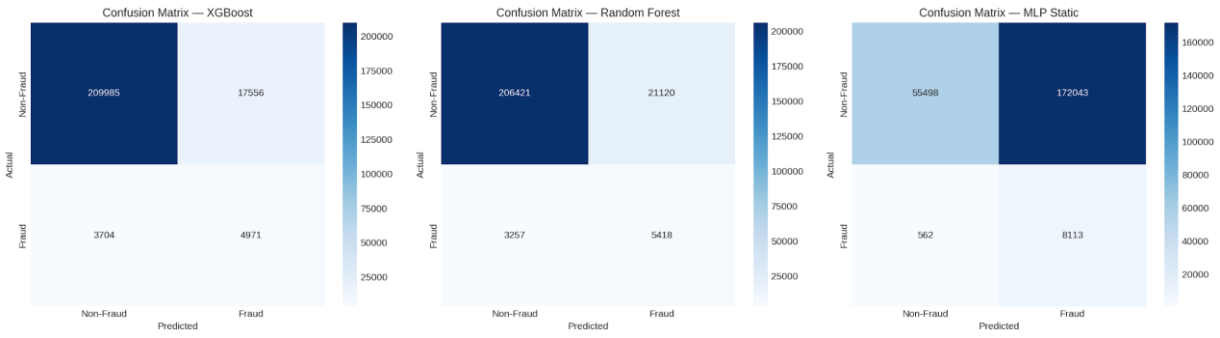

Figure 5.6: Confusion Matrices — XGBoost, Random Forest, MLP Static

At a standard 0.5 decision threshold, XGBoost correctly catches 4,971 fraud cases and raises 17,556 false alarms. Random Forest catches slightly more fraud (5,418) but also has more false alarms (21,120). The static MLP's numbers look very different: it catches 8,113 fraud cases but raises an enormous 172,043 false alarms. This means the MLP, when used

with a 0.5 threshold, would be practically useless in a real setting because investigators would be buried in false alarms. However, this does not mean the MLP is a bad model — it just needs its decision threshold adjusted to a value that balances fraud catches against false alarms more sensibly.

5.7 How the Adaptive Model Performed Over Time

Figure 5.7 tracks the adaptive neural network's performance as it processed the 236,216 test transactions one by one, updating itself along the way. The vertical red lines mark the moments when ADWIN detected that fraud patterns had changed.

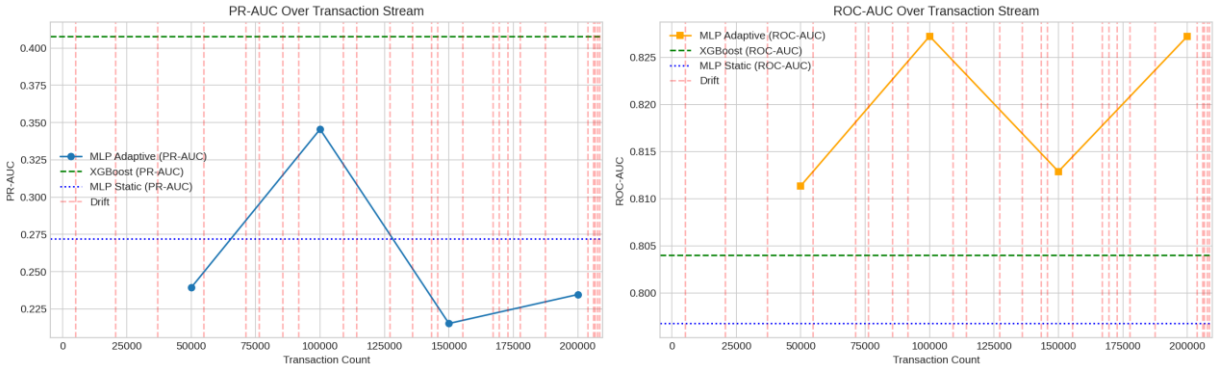

Figure 5.7: Adaptive MLP Performance Over Transaction Stream

The sheer number of drift detections throughout the stream confirms something important: fraud patterns in this dataset really do shift over time. The model's performance varies from chunk to chunk, peaking around the 100,000-transaction mark before dipping again. This variability is expected because in a real deployment, fraud labels only arrive with a delay, so the model is sometimes learning from stale information. On the ROC-AUC chart (right panel), the adaptive model actually surpasses both XGBoost and the static MLP during several stretches of the stream, particularly around transactions 90,000-110,000 and 190,000-210,000.

5.8 Key Takeaways

Pulling all the results together, the main lessons from this project are:

- Tree-based models (XGBoost, LightGBM, Random Forest) are the most reliable choice for fraud detection on structured transaction data, delivering strong scores across all three metrics without needing a GPU or complex training procedures.
- PR-AUC is a much more honest metric than ROC-AUC for this kind of problem. On PR-AUC, there is a clear gap between the tree-based models and the neural network models, which the ROC-AUC numbers do not fully show.
- Combining models through stacking gives a small but consistent boost in PR-AUC — from 41.3% for Random Forest alone to 43.0% for the Stacked Hybrid. This marginal gain might be worth the added complexity in a high-stakes production system.
- The adaptive neural network outperforms the static neural network in Recall@5%FPR, showing that adapting to changing fraud patterns does matter in practice.
- Logistic Regression is simply not powerful enough for this problem. Its scores are the lowest across all three metrics, confirming that drawing a single straight line through this data cannot capture the complexity of fraud patterns.
- The choice of decision threshold matters a lot. The static MLP's confusion matrix at a 0.5 threshold looks terrible, but its PR-AUC score (27.2%) shows that at the right threshold, it can perform much more sensibly.

CHAPTER 6: TESTING AND EVALUATION

6.1 Testing Strategy

A good evaluation strategy is just as important as having a good model. If you test your model carelessly, you might get results that look impressive on paper but fall apart in the real world. I designed the testing approach in this project with four principles in mind: keep the time order of transactions intact, prepare data using only training information, use the same random seeds every time so results can be reproduced, and never let any model peek at the test data during training.

6.2 Evaluation Protocol

6.2.1 Data Split

The dataset was split chronologically: the first 60% of transactions (354,324 records) for training and the last 40% (236,216 records) for testing. For the neural network, the training set was further divided 80/20 to create a small validation set used during training to monitor progress and avoid overfitting. All data transformation steps were fitted only on the training portion.

6.2.2 Reproducibility

To make sure anyone can reproduce these results, I fixed all random number generators to the same starting seed (42). This was applied in NumPy, PyTorch, scikit-learn, XGBoost, and LightGBM. With the same random seed and the same data, every run of the code should produce the same numbers.

6.2.3 How the Metrics Were Calculated

All three performance metrics were calculated on the full 40% test set:

- ROC-AUC was calculated using scikit-learn's roc_auc_score function, which compares the model's probability scores against the true labels.
- PR-AUC was calculated by first generating the full precision-recall curve with scikit-learn, then computing the area under it using the trapezoidal method.

- Recall@5%FPR was calculated by sorting all transactions from most to least suspicious according to the model, then counting how many actual fraud cases appear before the total number of false alarms exceeds 5% of all legitimate transactions.

6.3 Model Validation

6.3.1 Neural Network Early Stopping

To avoid training the neural network for too long and causing it to overfit, I monitored its PR-AUC score on the validation set after each training round. If the validation PR-AUC did not improve for 5 consecutive rounds, training stopped automatically and the best checkpoint was restored. This ensures the final model is the one that performed best on unseen validation data, not the one trained the longest.

6.3.2 Stacking Validation

For the Stacked Hybrid model, the three base models (Random Forest, XGBoost, MLP) were trained on the first 80% of the training data. Their predictions on the remaining 20% were used to train the combining Logistic Regression. This two-stage process ensures that the combining model is learning from predictions on data the base models have not seen, which prevents overly optimistic performance estimates.

6.4 Limitations

I want to be honest about the limitations of this testing setup:

- No confidence ranges: The scores reported are point estimates with no margin of error. Running the experiment 1,000 times with different random samples of the test set (a technique called bootstrapping) would give confidence intervals and allow proper statistical tests of whether one model is truly better than another.
- Limited hyperparameter tuning: I used sensible default settings for all models rather than searching for the very best settings. Tools like Optuna can automate this search and would likely squeeze out additional performance.
- Chunk-based streaming evaluation: The adaptive model's performance was measured in blocks of 50,000 transactions rather than transaction-by-transaction. This gives a rougher picture of how performance evolves over time.

- Single stacking fold: The combiner model was trained on predictions from a single held-out validation fold rather than using cross-validation, which would give a more stable estimate of its performance.

6.5 Software and Hardware

The entire project was built in Python 3.10 and run on Google Colab using a free T4 GPU provided by Google. The main libraries used were PyTorch for the neural network, scikit-learn for the classical models and evaluation tools, XGBoost, LightGBM, River for drift detection, and pandas for data handling. All trained models were saved to disk so they can be loaded and used in a web application without retraining.

CHAPTER 7: CONCLUSIONS AND FUTURE WORK

7.1 Conclusions

This project set out to answer a straightforward question: which machine learning model is best for detecting credit card fraud in a large, imbalanced dataset? After building, training, and testing eight different models on the IEEE-CIS Fraud Detection dataset, I have a clear set of answers.

The most important findings are:

- Tree-based ensemble models — particularly Random Forest and the gradient boosting methods XGBoost and LightGBM — are the most reliable and practical choices for structured fraud detection. They deliver strong results without requiring a GPU or complex tuning, and they can tell you which features matter most, making them easier to trust and explain to stakeholders.
- Combining multiple models into a Stacked Hybrid gives the best overall PR-AUC score (43.0%), which is the metric that matters most in fraud detection. The gain over using any single model is small but consistent, and it may be worth the extra effort in high-stakes applications.
- PR-AUC is a far more meaningful metric than ROC-AUC for imbalanced datasets. Looking only at ROC-AUC would give a misleadingly positive picture of models that actually struggle to find fraud without generating too many false alarms.
- Adapting to change matters. The Adaptive MLP caught more real fraud per false alarm than the Static MLP (Recall@5%FPR of 41.3% vs 39.2%), which shows that a model capable of updating itself as fraud patterns shift has a genuine advantage over one that is frozen after initial training.
- Logistic Regression is too simple for this problem. With the lowest scores across all three metrics, it is clear that a straight-line decision boundary cannot capture the complex patterns that distinguish fraud from legitimate transactions.
- Feature engineering is highly valuable. The most predictive feature by far was card1_fraud_rate — the historical fraud rate for the card being used. This was not in

the original data; it was created during the preprocessing step, highlighting how much human insight into the domain can boost model performance.

7.2 Future Work

There are many ways this project could be extended and improved:

- Richer features: Adding features like how many transactions a card makes per hour, how long since the last transaction, and the distance between consecutive transaction locations could help models catch fraud patterns that are invisible in the current feature set.
- Transformer models for tabular data: New architectures like TabNet and FT-Transformer apply the self-attention mechanism — made famous by ChatGPT — to structured data like transactions. These are worth exploring as alternatives to the MLP.
- Graph-based approaches: Fraud networks often involve the same card being used at many different merchants, or the same device being used with different cards. Modeling transactions as a graph and applying graph neural networks could uncover these network-level patterns.
- Real deployment testing: The final step would be to test the system in a real payment processing environment, measuring actual latency, scalability, and performance against live fraud.

7.3 Summary

This project showed that a well-designed machine learning pipeline — with careful data preparation, a range of model types, special handling for imbalanced data, and an adaptive component for drift — can achieve strong fraud detection performance on a realistic dataset. The Stacked Hybrid model delivered the best overall scores, but the most interesting finding was the adaptive neural network's ability to keep up with shifting fraud patterns in a live stream. The code, models, and results from this project are fully reproducible and provide a solid foundation for anyone looking to build on this work in the future.

REFERENCES

[1] A. Abdallah, M. A. Maarof, and A. Zainal, "Fraud detection system: A survey," *J. Netw. Comput. Appl.*, vol. 68, pp. 90–113, Jun. 2016, doi: 10.1016/j.jnca.2016.04.007.

[2] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448, doi: 10.1137/1.9781611972771.42.

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[6] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *2015 IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2015, pp. 159–166, doi: 10.1109/SSCI.2015.33.

[7] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," *Inf. Sci.*, vol. 479, pp. 448–455, Apr. 2019, doi: 10.1016/j.ins.2019.01.031.

[8] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 1050–1059.

[9] IEEE Computational Intelligence Society and Vesta Corporation, "IEEE-CIS Fraud Detection," Kaggle, 2019.

[10] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 3146–3154, doi: 10.5555/3294996.3295074.

[11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vision (ICCV)*, 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.324.

[12] C. Wang, J. Han, Y. Zhao, and Z. Yang, "Transaction fraud detection via an adaptive graph neural network," 2018, arXiv:1805.02322.

[13] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992, doi: 10.1016/S0893-6080(05)80023-1.